Automatically Generate More Operators

Document Number: P1046R0
Date: 2018-04-28
Author: David Stone (davidmstone@google.com)
Audience: Evolution Working Group (EWG)

Summary

This proposal follows the lead of `operator<=>` (see Consistent Comparison) by generating rewrite rules if a particular operator does not exist in current code.

- Rewrite `lhs += rhs` as `lhs = std::move(lhs) + rhs`
- Rewrite `lhs -= rhs` as `lhs = std::move(lhs) - rhs`
- Rewrite `lhs *= rhs` as `lhs = std::move(lhs) * rhs`
- Rewrite `lhs /= rhs` as `lhs = std::move(lhs) / rhs`
- Rewrite `lhs %= rhs` as `lhs = std::move(lhs) % rhs`
- Rewrite `lhs &= rhs` as `lhs = std::move(lhs) & rhs`
- Rewrite `lhs |= rhs` as `lhs = std::move(lhs) | rhs`
- Rewrite `lhs ^= rhs` as `lhs = std::move(lhs) ^ rhs`
- Rewrite `lhs <<= rhs` as `lhs = std::move(lhs) << rhs`
- Rewrite `lhs >>= rhs` as `lhs = std::move(lhs) >> rhs`
- Rewrite `++x` as `x += 1`
- Rewrite `--x` as `x -= 1`
- Rewrite `x++` as `++x` but return the original value
- Rewrite `x--` as `--x` but return the original value
- Rewrite `lhs->rhs` as `(*lhs).rhs`

Design Goals

There are certain well-defined patterns in code for how certain operators "should" be written. Many of these guidelines are widely followed and correct, leading to large amounts of code duplication. Some of these guidelines are widely followed and less optimal since C++11, leading to an even worse situation. Worst of all, some guidelines do not exist because the "correct" code cannot be written in C++. This proposal attempts reduce boilerplate, improve performance, increase regularity, and remove bugs.

One of the primary goals of this paper is that users should have very little reason to write their own version of an operator that this paper proposes generating. It would be a strong indictment if there are many examples of well-written code for which this paper provides no simplification, so a fair amount of time in this paper will be spent looking into the edge cases and making sure that we generate the right thing. At the very least, types which would not have the correct operator generated should not generate an operator at all. In other words, it should be uncommon for users to define their own versions (`= default` should be good enough most of the time), and it should be rare that users want to supress the generated version (`= delete` should almost never appear).

Arithmetic, Bitwise, and Compound Assignment Operators

For all binary arithmetic and bitwise operators, there are corresponding compound assignment operators. There are two obvious ways to implement these pairs of operators without code duplication: implement `a += b` in terms of `a = a + b` or implement `a = a + b` in terms of `a += b`. Using the compound assignment operator as the basis has a tradition going back to C++98, so let's consider what that code looks like using `string` as our example:

```
string & operator+=(string & lhs, string const & rhs) {
    lhs.append(rhs);
    return lhs;
}
string & operator+=(string & lhs, string && rhs) {
    if (lhs.capacity() >= lhs.size() + rhs.size()) {
        return lhs += std::as_const(rhs);
    } else {
        rhs.insert(0, lhs);
        lhs = std::move(rhs);
        return lhs;
    }
}

string operator+(string const & lhs, string const & rhs) {
    string result;
    result.reserve(lhs.size() + rhs.size());
    result = lhs;
    result += rhs;
    return result;
}
string operator+(string const & lhs, string && rhs) {
    rhs.insert(0, lhs);
    return std::move(rhs);
}
template<typename String> requires std::is_same<std::decay_t<String>, string>{}
string operator+(string && lhs, String && rhs) {
    lhs += std::forward<String>(rhs);
    return std::move(lhs);
}
```

This does not seem to save us as much as we might like. We can have a single template definition of `string + string` when we take an rvalue for the left-hand side argument, but when we have an lvalue as our left-hand side we need to write two more overloads (one of which cannot even use `operator+=`). It may seem tempting at first to take that approach, but taking advantage of move semantics eliminates most code savings. This does not feel like a generic answer to defining one class of operators.

What happens if we try to use `operator+` as the basis for our operations? The initial reason that C++03 code typically defined its operators in terms of `operator+=` was for efficiency, but with the addition of move semantics to C++11, we can regain this efficiency. What does the ideal `string + string` implementation look like?

```
string operator+(string const & lhs, string const & rhs) {
    string result;
    result.reserve(lhs.size() + rhs.size());
    result = lhs;
    result.append(rhs);
    return result;
}
string operator+(string && lhs, string const & rhs) {
    lhs.append(rhs);
    return std::move(lhs);
}
string operator+(string const & lhs, string && rhs) {
    rhs.insert(0, lhs);
    return std::move(rhs);
}
string operator+(string && lhs, string && rhs) {
    return (lhs.size() + rhs.size() <= lhs.capacity()) ?
        std::move(lhs) + std::as_const(rhs) :
        std::as_const(lhs) + std::move(rhs);
}

template<typename String> requires std::is_same<std::decay_t<String>, string>{}
string & operator+=(string & lhs, String && rhs) {
    lhs = std::move(lhs) + std::forward<String>(rhs);
```

```
    return lhs;
}
```

This mirrors the implementation if `operator+=` is the basis operation: we still end up with five functions in our interface. One interesting thing of note here is that the `operator+=` implementation does not know anything about its types other than that they are addable and assignable. Compare that with the `operator+` implementations from the first piece of code where they need to know about sizes, reserving, and insertion because `operator+=` alone is not powerful enough. There are several more counterexamples that argue against `operator+=` being our basis operation, but most come down to the same fact: `operator+` does not treat either argument specially, but `operator+=` does.

The first counterexample is `std::string` (again). We can perform `string += string_view`, but not `string_view += string`. If we try to define `operator+` in terms of `operator+=`, that would imply that we would be able to write `string + string_view`, but not `string_view + string`, which seems like an odd restriction. The class designer would have to manually write out both `+=` and `+`, defeating our entire purpose.

The second counterexample is the `bounded::integer` library. This library allows users to specify compile-time bounds on an integer type, and the result of arithmetic operations reflects the new bounds based on that operation. For instance, `bounded::integer<0, 10> + bounded::integer<1, 9> == bounded::integer<1, 19>`. If `operator+=` is the basis, it requires that the return type of an operation is the same as the type of the left-hand side of that operation. This is fundamentally another instance of the same problem with `string_view + string`.

The third counterexample is all types that are not assignable (for instance, if they contain a const data member). This restriction makes sense conceptually. An implementation of `operator+` requires only that the type is addable in some way. Thanks to guaranteed copy elision, we can implement it on most types in such a way that we do not even require a move constructor. An implementation of `operator+=` requires that the type is both addable and assignable. We should make the operations with the broadest applicability our basis.

Let's look a little closer at our `operator+=` implementation that is based on `operator+` and see if we can lift the concepts used to make the operation more generic. For strings, we wrote this:

```
template<typename String> requires std::is_same<std::decay_t<String>, string>{}
string & operator+=(string & lhs, String && rhs) {
    lhs = std::move(lhs) + std::forward<String>(rhs);
    return lhs;
}
```

The first obvious improvement is that the types should not be restricted to be the same type. Users expect that if `string = string + string_view` compiles, so does `string += string_view`. We are also not using anything string specific in this implementation, and our goal is to come up with a single `operator+=` that works for all types. With this, we end up with:

```
template<typename LHS, typename RHS>
LHS & operator+=(LHS & lhs, RHS && rhs) {
    lhs = std::move(lhs) + std::forward<RHS>(rhs);
    return lhs;
}
```

Are there any improvements we can make to this signature? One possible problem is that we unconditionally return a reference to the lhs, regardless of the behavior of the assignment operator. This is what is commonly done, but there are some use cases for not doing so. With `volatile` qualifiers, implementations diverge in whether returning a reference to an object qualifies as a read from `volatile`, and thus also diverge in behavior of warnings from ignoring a volatile-qualified result. This means that types which want to support volatile often return `void` from their assignment operator. A more compelling example might be `std::atomic`, which returns `T` instead of `atomic<T>` in `atomic<T> = T`, and this behavior is reflected in the current compound assignment operators. An improvement to our previous signature, therefore, is the following implementation:

```
template<typename LHS, typename RHS>
decltype(auto) operator+=(LHS & lhs, RHS && rhs) {
```

```
    return lhs = std::move(lhs) + std::forward<RHS>(rhs);
}
```

Now our compound assignment operator returns whatever the assignment operator returns. We have a technical reason to make this change, but does it match up with our expectations? Conceptually, `operator+=` should perform an addition followed by an assignment, so it seems reasonable to return whatever assignment returns, and this mirrors the (only?) type in the standard library that returns something other than `*this` from its assignment operator.

One final question might be about that somewhat strange looking `std::move(lhs)` piece. You might wonder whether this is correct or safe, since it appears to lead to a self-move-assignment. Current guidance from people such as Eric Niebler and Howard Hinnant is that self-move should be legal, but that such an expression does not necessarily leave the object in any paricular state. Fortunately, we sidestep this problem entirely becaise there is actually no self-move-assignment occuring here. For well-behaved types, `operator+` might accept its argument by rvalue-reference, but that function then returns by value. This means that the object assigned to `lhs` is a completely separate object, and there are no aliasing concerns.

Allocators

There is one major remaining question here, which is how this interacts with allocators. In particular, we might care about the values in `std::allocator_traits` for `propagate_on_container_copy_assignment`, `propagate_on_container_move_assignment`, `propagate_on_container_swap` and `select_on_container_copy_construction`. The move constructor of a standard container has no special interaction with allocators -- in particular, for all containers (except `std::array`) the move constructor is constant time, regardless of the type or value of the source allocator.

With an eye toward allocators, let's look again at the case of `string` and the operator overloads (with one minor modification to match the current specification of `operator+` with regards to allocators):

```
string operator+(string const & lhs, string const & rhs) {
    string result(allocator_traits::select_on_container_copy_construction(lhs.get_allocator()));
    result.reserve(lhs.size() + rhs.size());
    result.append(lhs);
    result.append(rhs);
    return result;
}
string operator+(string && lhs, string const & rhs) {
    lhs.append(rhs);
    return std::move(lhs);
}
string operator+(string const & lhs, string && rhs) {
    rhs.insert(0, lhs);
    return std::move(rhs);
}
string operator+(string && lhs, string && rhs) {
    return (lhs.size() + rhs.size() <= lhs.capacity()) ?
        std::move(lhs) + std::as_const(rhs) :
        std::as_const(lhs) + std::move(rhs);
}

template<typename LHS, typename RHS>
decltype(auto) operator+=(LHS & lhs, RHS && rhs) {
    return lhs = std::move(lhs) + std::forward<RHS>(rhs);
}
```

Our goal here is to make sure that given the proper definitions of `operator+`, our generic definition of `operator+=` will still do the correct thing. This is somewhat tricky, so we'll walk through this step by step. First, let's consider the case where `rhs` is anything but a `string &&` -- for this example, we will use `string const &`. We end up with this instantiation:

```
string & operator+=(string & lhs, strong const & rhs) {
    return lhs = std::move(lhs) + rhs;
}
```

That will call this overload of `operator+`:

```
string operator+(string && lhs, string const & rhs) {
    lhs.append(rhs);
    return std::move(lhs);
}
```

Before calling this function, nothing has changed with allocators because all we have done is bound variables to references. Inside the function, we make use of the allocator associated with `lhs`, if necessary, which is exactly what we want (the final object will be `lhs`). We then return `lhs`, which move constructs the return value. Move construction is guaranteed to move in the allocator, so our resulting value has the same allocator as `lhs` did at the start of the function. Going back up the stack, we then move assign this constructed string back into `lhs`. This will be fast because those allocators compare equal. This is true regardless of the value of `propagate_on_container_move_assignment`, since propagation is unecessary due to the equality.

The other case to consider is when `rhs` is `string &&`. Let's walk through the same exercise:

```
string & operator+=(string & lhs, strong && rhs) {
    return lhs = std::move(lhs) + std::move(rhs);
}
```

That will call this overload of `operator+`:

```
string operator+(string && lhs, string && rhs) {
    return (lhs.size() + rhs.size() <= lhs.capacity()) ?
        std::move(lhs) + std::as_const(rhs) :
        std::as_const(lhs) + std::move(rhs);
}
```

If `lhs` has enough capacity for the resulting object, we forward to the same overload we just established as being correct for our purposes, so we only need to consider the second branch. Unfortunately, in that second branch, we end up returning `rhs`, with its allocator. If `rhs` had enough capacity or if `propagate_on_container_move_assignment` is `true_type`, this is not such a big deal because we already established we needed to allocate more space in `lhs`, so we do not incur any additional allocations.

First, let's quantify the way in which there is a problem. There is a problem only in the case where `propagate_on_container_move_assignment` is `false_type`, the allocators compare unequal, there is insufficient capacity in the left-hand-side argument, and a temporary is passed in for the right-hand-side argument. In this case, we will first allocate enough space to hold the result in `rhs`, then allocate enough space to hold the result in `lhs`. This is twice as many allocations as necessary, which is a bad thing.

This means that, under certain allocator models (including the default `allocator_traits` model), allocator-aware types will still need to define their own compound assignment operators. Fortunately, the existence of overloaded operators on such types is rare (with `operator+` being used for string concatenation being seen by many as a mistake), and they are especially rare in environments that take advantage of stateful allocators, because `operator+` does not allow specifying the allocator of the resulting type.

Note that these examples still hold if we add in overloads for `string_view` and `char const *`. For simplicity, they have been left out.

Does this break the stream operators?

For "bitset" types, the expression `lhs << rhs` and `lhs >> rhs` mean something very different than other types, thanks to those operators being used for stream insertion / extraction. Does this proposal lead to the creation of a strange <<= or >>= operator with stream arguments?

Fortunately, it does not. The existing stream operators require the stream to start out on the left-hand side. This means that the only compound assignment operators that this proposal attempts to generate is are `stream <<= x` and `stream >>= x`. For this to compile using `std::istream` or `std::ostream`, the user must have overloaded `operator>>` or `operator<<` to return something other than the standard of `stream &` and instead return something that can be used as the source of an assignment to a stream object. There are not many types that meet this critera, so the odds of this happening accidentally are slim.

Recommendation

We should "generate" compound assignment operators following the same rules as we follow for `operator<=>`: if an expression contains a call to a compound assignment operator and there is no such user-declared operator, the expression is instead rewritten to be the decomposed version: `lhs @= rhs` becomes `lhs = std::move(lhs) @ rhs`. Users can opt-in to using such an operator rewrite (rather than using it only as a fallback) by specifying `= default`. They can opt-out using `= delete`. This proposal would "generate" rewrite rules for the following operators:

- `operator+=`
- `operator-=`
- `operator*=`
- `operator/=`
- `operator%=`
- `operator&=`
- `operator|=`
- `operator^=`
- `operator<<=`
- `operator>>=`

Prefix Increment and Decrement

Recommendation

Rewrite `++a` as `a += 1`, and rewrite `--a` as `a -= 1`. This will even match the current behavior for `std::atomic`.

Postfix Increment and Decrement

Recomendation

Rewrite `a++` as something like the statement expression `({ auto __temp = a; ++a; __temp; })` and rewite `a--` as something like the statement expression `({ auto __temp = a; --a; __temp; })`. This will work for most normal types, but fails to compile for `std::atomic` because `std::atomic` is non-copyable. This is fine, because `std::atomic` would need to provide different functionality by returning a copy of the original `T`, rather than a copy of the original `std::atomic<T>`.

Member of pointer (->)

There appear to be a few design decisions to make for this operator: how do we get the address of the object, and how do we handle temporaries. We will discuss some possible approaches, and hopefully come up with a solution that avoids their drawbacks.

First, there are two ways to get the address:

```
auto operator->() const {
    return std::addressof(operator*());
```

```
}
```

or

```
auto operator->() const {
    return &(operator*());
}
```

The first version guarantees that we always return the address of the object returned by `operator*`, so `operator->` will likely have the behavior expected by the caller. The second version assumes that if unary `operator&` was overloaded, it must have been for some reason and the returned type must be doing something important, so it has the behavior likely expected by the writer of the type returned by `operator*`. The general existing practice in generic code is to always use the "true" address by calling `addressof`, and the general recommendation in broader C++ is to not overload unary `operator&`.

Second, do we attempt to make types which return a temporary from `operator*` work, using something like

```
struct temporary_operator_arrow {
    using value_type = std::remove_reference_t<decltype(operator*())>;
    explicit temporary_operator_arrow(value_type value):
        m_value(std::move(value))
    {
    }
    auto operator->() && {
        return std::addressof(m_value);
    }

private:
    value_type m_value;
};

auto operator->() const {
    if constexpr (returns temporary) {
        return temporary_operator_arrow(operator*());
    } else {
        return std::addressof(operator*());
    }
}
```

Doing this indirection layer has the advantage that code like `f(it->x)` would just work, even if `*it` returns by value (as long as `*it` is movable). The downside of this approach is that it silently breaks user expectations around lifetime extension with references. In other words, if `*it` returns a temporary, the following code compiles and uses an object after its lifetime has ended:

```
auto && ref = it->x;
f(ref);
```

These would be the possible approaches if we were to design a library-only solution. However, this paper traffics in rewrite rules (following the lead of `operator<=>`), not in terms of function calls. Because of this, we have one more option: we could define `lhs->rhs` as being equivalent to `(*lhs).rhs` (assuming lhs does not have an `operator->` defined). This neatly sidesteps all of the previous discussion by simply making the questions unnatural to ask, with the results implied by existing language rules. It even plays nicely with existing rules around lifetime extension of temporaries.

Recommendation

Rewrite `lhs->rhs` as `(*lhs).rhs`.

Subscript (`[]`)

The subscript operator is a bit trickier. The problem here is that there are actually at least three different subscript operators with the same syntax. The first subscript operator is the subscript operator of iterators: `it[n]` is equivalent to `*(it + n)`. The second subscript operator is the subscript operator of random-access ranges: `range[n]` is equivalent to `begin(range)[n]`. The third subscript operator is lookup into an associative container: `dictionary[key]` is equivalent to `*dictionary.emplace(key).first`. This gets even more complicated when you consider that a type can model both a random-access range and an associative container (for instance: `flat_map`).

Recommendation

Do nothing.

`operator-`

`a - b` is theoretically equivalent to `a + -b`. This is true in C++ only for types which accurately model their mathematical equivalents or else maintain a modular equivalence (for instance, unsigned types). The problem is that the expression `-b` might overflow. Even if that negation is well defined, there still may be overflow; consider `ptr - 1U`: if we rewrote that to be `ptr + -1U`, the pointer arithmetic would overflow. Attempting this rewrite is a dangerous non-starter.

How about the other way around? Perhaps we could define `-a` as `0 - a`? This could be a promising avenue, but it is not proposed in this paper. That exact definition would need some tweaking, as this would give pointers a unary minus operator with undefined behavior for non-null pointers (`0` is also a null pointer constant, allowing the conversion, and pointer arithmetic is undefined for out-of-bounds arithmetic, but this is a solvable problem).

Recommendation

Do nothing.